instituto superior de engenharia de lisboa

Departamento de Engenharia de Electrónica e Telecomunicações e de Computadores

Primeiro Trabalho Prático

51445 : José Saldanha (a51445@alunos.isel.pt)

51447 : Ricardo Pinto (a51447@alunos.isel.pt)

51484 : Vasco Pereira (a51484@alunos.isel.pt)

Relatório para a Unidade Curricular de Segurança Informática
da Licenciatura em Engenharia Informática e de Computadores

Professor : Dr. José Simão

23 – outubro – 2025

Contents

1 Exercício 1 **1**

1.1 Tipos de proteção e operações criptográficas 1

1.2 Modos de operação e diferenças 1

1.3 Desproteger e verificar 2

2 Exercício 2 **5**

2.1 Implementação da comunicação 5

2.2 Análise de confidencialidade e integridade 6

3 Exercício 3 **7**

3.1 Cadeia de certificados do site example.org 7

3.2 Algoritmo de assinatura do certificado 7

4 Exercício 4 **9**

4.1 Função de hash H16 9

4.2 Implementação em Java 9

4.3 Resultados e ambiente de teste 10

4.4 Consequências para SHA-256 completo 10

5 Exercício 5 **11**

5.1 Cifra autenticada e estabelecimento de chaves 11

5.1.1 Descrição geral da implementação 11

5.1.2 Geração aleatória da chave 12

5.1.3 Processo de cifra autenticada 12

5.1.4 Processo de decifra e verificação 12

5.1.5 Comandos e execução 13

5.2 Estabelecimento de chaves sem canal seguro 13
5.2.1 Esquema híbrido de troca de chaves 13
5.2.2 Modificação à definição (1) 14
5.2.3 Vantagens do esquema híbrido 14

6 Exercício 6 **15**
6.1 Assinatura digital com RSA 15
6.1.1 Descrição geral da implementação 15
6.1.2 Modo de assinatura (`-sign`) 15
6.1.3 Modo de verificação (`-verify`) 16
6.1.4 Processamento de ficheiros de grande dimensão 16
6.1.5 Validação da cadeia de certificação 17
6.1.6 Execução e utilização 17
6.1.7 Conclusão 17

1

Exercício 1

1.1 Tipos de proteção e operações criptográficas

Os comandos aplicam dois tipos de proteção:

- **Confidencialidade**, através de cifra simétrica `AES-128-CBC`;
- **Autenticidade e integridade**, através de um código de autenticação de mensagem `HMAC-SHA256`.

Usando a notação dos slides:

$$\begin{cases} c = E(k_1)(m) = AES\text{-}128\text{-}CBC_{k_1}(f) \\ t = T(k_2)(m) = HMAC\text{-}SHA256_{k_2}(f) \end{cases}$$

1.2 Modos de operação e diferenças

O primeiro comando OpenSSL:

```
$ openssl enc -e -aes-128-cbc -in f -out g \
-iv 0ed425637bb85abc9790850627b2355d -K 42c2771f807215802dff1329fa7bf2cf
```

realiza a **cifra simétrica** do ficheiro `f`, produzindo o ficheiro cifrado `g`. É utilizado o algoritmo `AES-128` no modo de operação `CBC` (*Cipher Block Chaining*), o qual cifra em blocos de tamanho fixo.

De acordo com os slides, as operações do modo CBC são descritas por:

$$c_i = E_k(m_i \oplus c_{i-1}), \quad c_0 = IV$$

$$m_i = D_k(c_i) \oplus c_{i-1}$$

Cada bloco de texto cifrado depende do anterior, criando uma cadeia que dificulta a deteção de padrões. Um erro num bloco c_i afeta a decifra do próprio bloco e do seguinte c_{i+1}, o que garante segurança adicional, mas impossibilita o acesso aleatório aos blocos.

Se, em alternativa, fosse utilizado o modo `CTR` (*Counter Mode*), através do comando:

```
$ openssl enc -e -aes-128-ctr -in f -out g -K ... -iv ...
```

então o AES passaria a operar em modo *stream*, tal como indicado nos slides 28–29 do powerpoint sobre esquemas primitivos simétricos. O modo CTR gera um fluxo de chaves (keystream) a partir de um contador:

$$c_i = m_i \oplus E_k(I_i), \quad I_i = IV + i$$

$$m_i = c_i \oplus E_k(I_i)$$

Neste modo, cada bloco é cifrado independentemente, permitindo paralelismo e acesso aleatório à informação cifrada. A propagação de erros também é diferente: um erro em c_i afeta apenas m_i, não comprometendo os blocos seguintes.

Em resumo:

- O modo **CBC** é um modo de cifra em *blocos*, dependente dos blocos anteriores, onde um erro em c_i afeta m_i e m_{i+1}.
- O modo **CTR** é um modo de cifra em *stream*, em que os blocos são independentes e um erro em c_i afeta apenas m_i.

1.3 Desproteger e verificar

O segundo comando OpenSSL:

```
$ openssl mac -digest SHA256 -macopt key:changeit -in f -out h HMAC
```

gera o **código de autenticação de mensagem (MAC)** do ficheiro `f`, guardando o resultado no ficheiro `h`. O algoritmo utilizado é o `HMAC-SHA256`, um esquema MAC que garante **integridade** e **autenticidade** da mensagem. Formalmente:

$$t = T(k_2)(m) = HMAC\text{-}SHA256_{k_2}(f)$$

Após este passo, os ficheiros `g` (texto cifrado) e `h` (tag HMAC) são concatenados num único ficheiro `result`, combinando confidencialidade e autenticidade de acordo com o esquema *Encrypt-then-MAC* apresentado nos slides:

$$E(k_1)(M)||T(k_2)(E(k_1)(M))$$

O processo de **desproteção e verificação** é então o inverso:

1. Separar o ficheiro `result` nos seus dois componentes:

$$g = \text{parte cifrada}, \quad h = \text{tag HMAC}$$

2. Decifrar o ficheiro cifrado:

$$f = D(k_1)(g)$$

   A decifra usa o mesmo algoritmo e parâmetros (chave e IV) usados na cifra.

3. Verificar a autenticidade e integridade:

$$v = V(k_2)(h, f)$$

   O HMAC é recalculado sobre `f` e comparado com `h`.

4. Se $v =$ true, conclui-se que a mensagem é autêntica e íntegra; caso contrário, houve alteração ou corrupção dos dados.

Deste modo:

- O ficheiro `g` garante **confidencialidade** (cifra AES-128-CBC);
- O ficheiro `h` garante **autenticidade e integridade** (HMAC-SHA256);
- O ficheiro `result` combina ambos, permitindo a posterior **verificação e decifra seguras**.

2

Exercício 2

2.1 Implementação da comunicação

Pretende-se implementar o envio e receção de uma mensagem entre **Alice** e **Bob**, usando um esquema de **cifra/decifra assimétrica** e a função de hash `SHA256`. A chave pública de Bob (`KeBob`) é usada por Alice para cifrar a mensagem, e a chave privada de Bob (`KdBob`) é usada para a decifrar.

Envio (Alice)

```
# Cifra a mensagem com a chave pública de Bob
openssl rsautl -encrypt -inkey KeBob.pem -pubin -in m -out c

# Calcula o hash SHA256 da mensagem original
openssl dgst -sha256 -binary m > h

# Concatena ambos os ficheiros
cat c h > ch
```

Assim, o ficheiro `ch` contém o criptograma e o hash:

$$ch = E_a(Ke_{Bob})(m)||SHA256(m)$$

Receção (Bob)

```
# Divide o ficheiro recebido
head -c <tam_c> ch > c
tail -c 32 ch > h
```

```
# Decifra com a chave privada de Bob
openssl rsautl -decrypt -inkey KdBob.pem -in c -out m

# Recalcula o hash da mensagem decifrada
openssl dgst -sha256 -binary m > h2

# Compara os hashes
cmp h h2 && echo "Mensagem íntegra" || echo "Falha de integridade"
```

No final do processo, Bob obtém a mensagem original m e verifica se o hash corresponde ao valor enviado.

2.2 Análise de confidencialidade e integridade

Confidencialidade

A cifra assimétrica garante **confidencialidade**, pois a mensagem é cifrada com a chave pública de Bob:

$$m = D_a(Kd_{Bob})(c)$$

Apenas Bob, possuindo a chave privada correspondente, consegue decifrar a mensagem. Um atacante que intercepte o criptograma c não consegue recuperar m sem conhecer Kd_{Bob}.

Integridade e autenticidade

O hash SHA256 permite verificar se a mensagem foi alterada após a transmissão. Contudo, este esquema **não garante integridade de forma robusta** nem autenticidade de Alice:

- O hash é enviado em claro juntamente com o criptograma:

  $$ch = E_a(Ke_{Bob})(m) \mid\mid SHA256(m)$$

  Um atacante poderia substituir o criptograma c e criar um novo par (c', h'), o que poderia induzir Bob a aceitar uma mensagem alterada.

- Para garantir **integridade** e **autenticidade**, recomenda-se que:
  - o hash seja protegido com uma chave secreta (por exemplo, usando HMAC), ou
  - a mensagem seja assinada digitalmente por Alice:

    $$s = S(Kd_{Alice})(SHA256(m))$$

    permitindo que Bob verifique a assinatura:

    $$V(Ke_{Alice})(s, SHA256(m)) = \text{true}$$

3

Exercício 3

3.1 Cadeia de certificados do site example.org

Ao aceder ao site `https://www.example.org`, verificamos que a cadeia de certificados é composta por três elementos:

1. **Certificado do servidor:** `*.example.org`, emitido pela *Internet Corporation for Assigned Names and Numbers (ICANN)*.
2. **Certificado intermédio:** `DigiCert Global G3 TLS ECC SHA384 2020 CA1`, emitido pela autoridade de certificação DigiCert.
3. **Certificado raiz:** `DigiCert Global Root G3`, autoassinado e presente no armazenamento de autoridades de certificação confiáveis dos sistemas operativos e navegadores.

A cadeia garante que o certificado do servidor é confiável, desde que a autoridade raiz seja confiável no sistema do cliente.

3.2 Algoritmo de assinatura do certificado

Usando o comando OpenSSL:

```
openssl s_client -connect example.org:443 -showcerts
```

é possível visualizar o campo `sigalg`, que indica o algoritmo usado para assinar o certificado:

```
sigalg: ecdsa-with-SHA384
```

Portanto, o certificado foi assinado utilizando o algoritmo **ECDSA (Elliptic Curve Digital Signature Algorithm)** em conjunto com a função de hash **SHA-384**. Isto garante a integridade do certificado e permite verificar que não foi alterado após a assinatura pela autoridade certificadora.

4

Exercício 4

4.1 Função de hash H16

Considera-se a função de hash $H16(m)$ definida a partir do SHA-256, mas utilizando apenas os primeiros 16 bits do hash:

$$H16(m) = SHA256(m)_{0..15}.$$

Dois programas com códigos-fonte m e m' são considerados equivalentes ($m \equiv m'$) se a sua execução produz o mesmo resultado observável.

O objetivo é encontrar um código-fonte m' tal que:

$$H16(m') = H16(m_2) \quad \text{e} \quad m' \equiv m_1.$$

4.2 Implementação em Java

Para realizar o ataque de colisão parcial, foi implementada uma aplicação Java usando a biblioteca JCA:

- `H16.java`: implementa a função $H16$ a partir do SHA-256, retornando apenas os dois primeiros bytes.
- `CollisionFinder.java`: lê o código de `BadApp.java`, insere um nonce incremental, e testa se o $H16$ coincide com o de `GoodApp.java`. Quando encontra uma colisão, escreve o candidato em `BadApp_candidate.java`.

Resumo do algoritmo

1. Lê o ficheiro `GoodApp.java` e calcula $H16(m_2)$.

2. Itera sobre nonces, inserindo-os como comentário no código `BadApp.java`.
3. Calcula $H16(m')$ do candidato.
4. Se $H16(m') = H16(m_2)$, grava o ficheiro e termina.

4.3 Resultados e ambiente de teste

O programa foi executado em três máquinas com características diferentes, realizando três medições em cada uma:

- **Laptop José** – Windows 11, Intel i7-12700H, RTX 4070 Laptop, 16GB RAM Tempos (segundos): 3.075, 3.036, 3.016 Média: 3.042 s
- **Laptop Vasco** – Windows 11, Intel i5-1135G7, IGPU, 16GB RAM Tempos (segundos): 4.600, 6.200, 5.147 Média: 5.316 s
- **Laptop Ricardo** – Windows 11, Intel i7-1255U, IGPU, 16GB RAM Tempos (segundos): 6.456, 6.063, 5.966 Média: 6.162 s

Em média, a colisão foi encontrada após aproximadamente **11 427 tentativas**, confirmando que a função $H16$ é vulnerável a ataques de colisão devido ao reduzido tamanho do hash (16 bits).

4.4 Consequências para SHA-256 completo

Se fosse possível realizar este tipo de ataque sobre os 256 bits do SHA-256, as consequências seriam graves para esquemas de assinatura digital:

- Um atacante poderia criar mensagens diferentes que gerassem o mesmo hash, permitindo falsificação de assinaturas digitais.
- A segurança de integridade e autenticidade oferecida por SHA-256 seria comprometida.
- No entanto, devido ao tamanho do hash (256 bits), ataques de colisão completos são computacionalmente impraticáveis atualmente.

5

Exercício 5

5.1 Cifra autenticada e estabelecimento de chaves

O exercício 5.1 propõe a implementação de um esquema de cifra autenticada simétrica utilizando a biblioteca Java Cryptography Architecture (JCA). O objetivo é desenvolver uma aplicação capaz de cifrar e decifrar ficheiros garantindo simultaneamente **confidencialidade** e **autenticidade**.

A cifra autenticada segue o modelo:

$$AE(k)(m) = E(k)(m) \parallel T(k)(E(k)(m))$$

onde:

- E é a função de cifra simétrica (AES em modo CBC com *padding* PKCS#5);
- T é a função de MAC (HMAC-SHA256);
- k é a chave secreta composta por duas partes (chave AES e chave HMAC);
- m representa a mensagem original.

5.1.1 Descrição geral da implementação

A aplicação `AEFile.java` implementa as funcionalidades necessárias de uma cifra autenticada. O programa aceita três modos de execução:

- `-genkey`: geração de chaves simétricas;
- `-cipher`: cifra e autenticação de um ficheiro;
- `-decipher`: verificação de autenticidade e decifra.

A estrutura foi desenhada para funcionar com ficheiros de qualquer dimensão e processar dados em modo *stream*, garantindo eficiência e escalabilidade.

5.1.2 Geração aleatória da chave

A função `generateKeyFile` cria um ficheiro de chaves em Base64 com duas linhas:

- A primeira contém a chave AES de 256 bits para confidencialidade;
- A segunda contém a chave HMAC de 256 bits para autenticidade.

As chaves são geradas de forma aleatória com `SecureRandom` e `KeyGenerator`, assegurando que cada execução produz chaves únicas e imprevisíveis.

5.1.3 Processo de cifra autenticada

O método `encryptFile` implementa o processo *encrypt-then-MAC*, considerado seguro e recomendado. O fluxo é o seguinte:

1. Gera-se um vetor de inicialização (IV) de 16 bytes.
2. Inicializa-se o cifrador AES em modo CBC-PKCS#5 e o HMAC-SHA256.
3. Os dados do ficheiro são lidos por blocos, cifrados e passados pela função de MAC simultaneamente.
4. O resultado final é o concatenado `IV || ciphertext || tag` codificado em Base64 e gravado no ficheiro de saída.

A inclusão do IV no cálculo do HMAC garante a integridade também do vetor de inicialização, protegendo contra ataques de substituição.

5.1.4 Processo de decifra e verificação

A função `decryptFile` realiza a decifra de forma controlada e segura:

1. O ficheiro é descodificado de Base64 para obter o `IV`, o `ciphertext` e o `tag`.
2. O HMAC é recalculado sobre `IV || ciphertext` e comparado com o `tag` original.
3. Se o HMAC for válido, a mensagem é decifrada com AES/CBC e gravada no ficheiro de saída.
4. Caso contrário, a decifra não é realizada e o programa informa que a mensagem não é autêntica.

Esta verificação assegura que modificações acidentais ou maliciosas no ficheiro são detetadas antes da decifra, preservando tanto a integridade como a confidencialidade dos dados.

5.1.5 Comandos e execução

A aplicação aceita três comandos principais:

- `-genkey <keyfile>` — gera um ficheiro de chaves simétricas;
- `-cipher <inputFile> <keyfile> <outputBase64File>` — cifra e autentica o ficheiro de entrada;
- `-decipher <inputBase64File> <keyfile> <outputFile>` — verifica a autenticidade e decifra o ficheiro.

Este esquema garante um fluxo de operação seguro e modular, com clara separação entre as responsabilidades de cifra, autenticação e verificação.

5.2 Estabelecimento de chaves sem canal seguro

Na alínea anterior, a chave simétrica k era armazenada num ficheiro e teria de ser transmitida por um canal seguro entre as partes. Para eliminar essa necessidade, é possível recorrer a um **esquema híbrido** que combina criptografia assimétrica com simétrica, permitindo trocar a chave de forma segura mesmo através de canais não confiáveis.

5.2.1 Esquema híbrido de troca de chaves

Neste modelo, cada participante possui um par de chaves assimétricas:

- Uma **chave pública** (PK), que pode ser partilhada livremente;
- Uma **chave privada** (SK), que deve ser mantida secreta.

A comunicação inicia-se quando o remetente (A) quer enviar uma mensagem confidencial ao destinatário (B). O processo segue os seguintes passos:

1. O remetente A gera uma chave simétrica temporária k (usada para AES e HMAC);
2. Cifra a mensagem m de acordo com o esquema autenticado:
$$C = E_k(m) \parallel T_k(E_k(m))$$
3. Cifra a chave simétrica k utilizando a **chave pública** do destinatário PK_B:
$$k_{enc} = E_{PK_B}(k)$$
4. Envia o conjunto:
$$AE_{hyb}(m) = k_{enc} \parallel C$$

5. O destinatário B, ao receber a mensagem, utiliza a sua **chave privada** SK_B para recuperar a chave simétrica:

$$k = D_{SK_B}(k_{enc})$$

6. Finalmente, usa k para decifrar e verificar a autenticidade de C.

Assim, a chave simétrica nunca é transmitida em texto simples e apenas o destinatário que possui a chave privada correspondente pode recuperá-la.

5.2.2 Modificação à definição (1)

A definição original do esquema autenticado era:

$$AE(k)(m) = E(k)(m) \parallel T(k)(E(k)(m))$$

No caso do esquema híbrido, esta definição é estendida para incluir a proteção da chave simétrica com criptografia assimétrica:

$$AE_{hyb}(PK_B)(m) = E_{PK_B}(k) \parallel \big(E_k(m) \parallel T_k(E_k(m))\big)$$

onde:

- $E_{PK_B}(k)$ é a cifra assimétrica da chave simétrica com a chave pública do destinatário;
- $E_k(m)$ e $T_k(E_k(m))$ representam, respetivamente, a cifra e o código de autenticação (MAC) da mensagem usando a chave simétrica k.

5.2.3 Vantagens do esquema híbrido

- **Elimina a necessidade de um canal seguro:** a chave simétrica é protegida pela cifra assimétrica.
- **Desempenho eficiente:** a criptografia simétrica é usada para os dados (rápida), e a assimétrica apenas para proteger a pequena chave k.
- **Confidencialidade garantida:** apenas quem possui a chave privada correspondente pode recuperar k.
- **Flexibilidade:** permite o uso de diferentes pares de chaves assimétricas sem alterar o algoritmo de cifra autenticada.

6

Exercício 6

6.1 Assinatura digital com RSA

O exercício 6 propõe a implementação de um esquema de assinatura digital baseado no algoritmo assimétrico **RSA**, usando a biblioteca *Java Cryptography Architecture (JCA)*. A aplicação permite assinar e verificar ficheiros de qualquer dimensão, suportando as funções de hash `SHA-1` e `SHA-256`, e valida a cadeia de certificação do certificado do assinante antes de verificar a assinatura.

6.1.1 Descrição geral da implementação

A aplicação `DigitalSign.java` define duas operações principais configuradas por linha de comandos: o modo de assinatura (`-sign`) e o modo de verificação (`-verify`). O algoritmo de assinatura selecionado depende da flag de hash: `SHA1withRSA` para `-sha1` e `SHA256withRSA` para `-sha256`.

A implementação faz uso das APIs padrão de segurança do Java para:

- Carregar *keystores* (formato PKCS#12) e extrair chaves privadas;
- Criar e utilizar objetos `Signature` para assinar/verificar dados;
- Carregar certificados X.509 e construir/validar caminhos de certificação com `CertPathBuilder` e parâmetros PKIX.

6.1.2 Modo de assinatura (`-sign`)

No modo de assinatura, o programa executa os seguintes passos (descritos conceptualmente, sem apresentar código):

1. O *keystore* em formato PKCS#12 é carregado utilizando a palavra-passe fornecida pelo utilizador.

2. O programa seleciona automaticamente a primeira entrada de chave privada encontrada no keystore. É feita uma verificação para garantir que a entrada obtida é efectivamente uma chave privada.
3. É inicializado um objecto de assinatura com o algoritmo apropriado (SHA-1 ou SHA-256 com RSA).
4. O ficheiro a assinar é lido de forma incremental, em blocos de dimensão fixa (no código de referência usa-se um buffer de 8 192 bytes), e cada bloco é fornecido ao objecto de assinatura para atualizar o seu estado de forma incremental.
5. Quando todo o ficheiro foi processado, é gerada a assinatura final (um array de bytes) que é gravada num ficheiro de saída dedicado.

A leitura incremental e a actualização por blocos do objecto de assinatura permitem que ficheiros de tamanho arbitrário sejam assinados sem carregar o conteúdo inteiro na memória.

6.1.3 Modo de verificação (`-verify`)

No modo de verificação, a aplicação procede conceptualmente assim:

1. Carrega o certificado do assinante (formato X.509).
2. Carrega o *truststore* (formato JKS) que contém as entidades certificadoras consideradas confiáveis.
3. Carrega certificados intermediários a partir de uma directoria configurada, para auxiliar na construção da cadeia.
4. Constrói e valida a cadeia de certificação do certificado do assinante usando o `CertPathBuilder` e parâmetros PKIX. Caso a cadeia não seja válida, o processo de verificação da assinatura é interrompido e sinaliza-se a falha de confiança.
5. Se a cadeia for válida, inicializa-se o objecto de verificação com a chave pública do certificado do assinante.
6. O ficheiro cujo conteúdo se pretende verificar é lido incrementalmente (blocos de 8 192 bytes) e cada bloco é fornecido ao objecto de verificação para actualizar o estado. No fim, a assinatura armazenada é comparada com o resultado calculado para determinar se a assinatura é válida.

A validação da cadeia antes da verificação assegura que apenas certificados confiáveis são usados para aceitar assinaturas.

6.1.4 Processamento de ficheiros de grande dimensão

A característica fundamental que permite ao programa processar ficheiros de qualquer dimensão é o processamento em streaming (leitura incremental com buffer):

- Em vez de carregar todo o ficheiro na memória, o programa lê blocos de tamanho fixo (no código de referência: 8 192 bytes) e processa cada bloco imediatamente.

- Nas operações de assinatura e de verificação, os métodos `update()` do objecto `Signature` são chamados por cada bloco lido, permitindo que o cálculo do digest/assinatura seja efectuado de forma incremental.
- Só é mantido em memória um pequeno buffer por cada iteração; o resto do ficheiro permanece em disco até ser lido. Isto garante baixa utilização de memória mesmo para ficheiros muito grandes.

Esta abordagem torna a aplicação escalável e apropriada para ficheiros desde alguns kilobytes até gigabytes, dependendo apenas das capacidades de I/O do sistema e não da RAM disponível.

6.1.5 Validação da cadeia de certificação

A validação da cadeia é tratada através das seguintes decisões de design (descritas conceptualmente):

- O certificado do assinante é utilizado como o ponto de partida (end-entity certificate).
- Certificados intermédios são adicionados a um repositório temporário que auxilia a construção do caminho até a(s) raiz(es) de confiança presentes no *truststore*.
- O mecanismo de construção e validação usa o modelo PKIX: são definidos parâmetros que incluem o *truststore* como fonte de raízes confiáveis e o certificado do assinante como alvo, e é invocado um `CertPathBuilder` para tentar construir e validar um caminho válido.
- Caso a construção falhe (cadeia incompleta, certificados expirados, ou outros erros de validação), o programa rejeita o certificado e reporta a cadeia como inválida, evitando assim verificar assinaturas com certificados não confiáveis.

6.1.6 Execução e utilização

A aplicação é controlada por argumentos na linha de comandos, distinguindo o modo de assinatura do modo de verificação e a função de hash a empregar. Em termos gerais, o fluxo de operação é:

- No modo de assinatura: fornecer ficheiro a assinar, *keystore*, palavra-passe e o ficheiro onde guardar a assinatura.
- No modo de verificação: fornecer ficheiro original, ficheiro de assinatura, certificado do assinante, *truststore* e password para o *truststore*; o programa informa se a assinatura é válida e se a cadeia de certificação foi reconhecida como confiável.

6.1.7 Conclusão

A implementação apresentada garante os requisitos fundamentais solicitados:

- Assinatura e verificação de ficheiros de qualquer dimensão, graças ao processamento incremental;
- Suporte a funções de hash adequadas para assinatura com RSA;

- Validação da cadeia de certificação antes da verificação da assinatura, assegurando confiança na origem.
- Seleção automática da chave privada no keystore, simplificando a utilização sem necessidade de fornecer um alias.